

Learning UML Database Design and Modeling with AutoER

Sarah Foss
sfoss@okanagan.bc.ca
University of British Columbia

Tatiana Urazova
turazova@student.ubc.ca
University of British Columbia

Ramon Lawrence
ramon.lawrence@ubc.ca
University of British Columbia

ABSTRACT

Designing models for software systems is a complex activity that takes time and practice to master. Supporting students learning design is time-consuming, and instructors are limited on the amount of feedback they can provide. This work demonstrates a learning system that helps students learn UML modeling to produce database designs. Important features of the system are its ability to auto-evaluate design diagrams based on instructor criteria, and automatically generate question variants allowing students to practice on their own. The system is integrated into the PrairieLearn learning management system allowing for its deployment at other institutions. The demonstration shows the random question generation, how students construct database designs, and the auto-marking capabilities. The system and practice questions are publicly available for instructors to use to help students learn design modeling.

CCS CONCEPTS

• **Social and professional topics** → **Student assessment; Information systems education; Software engineering education;**
• **Applied computing** → **Interactive learning environments; E-learning;**
• **Software and its engineering** → **Software design engineering;**
• **Information systems** → **Entity relationship models.**

KEYWORDS

UML, database design, models, education, automatic marking, auto-grading, assessment

ACM Reference Format:

Sarah Foss, Tatiana Urazova, and Ramon Lawrence. 2022. Learning UML Database Design and Modeling with AutoER. In *ACM/IEEE 25th International Conference on Model Driven Engineering Languages and Systems (MODELS '22 Companion)*, October 23–28, 2022, Montreal, QC, Canada. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3550356.3559091>

1 INTRODUCTION

Learning design and modeling is one of the most challenging tasks for students and requires substantial practice. Modeling requires familiarity with the underlying constructs of the modeling language, such as UML [13], and the ability to translate the semantics of the problem domain into the appropriate model. Students often produce designs with structural and semantic issues, and do not

use modeling constructs properly. Instructors are limited on the time that they can provide for feedback to help students learn.

This work demonstrates a system for teaching database design by automatically generating and marking questions. The database design problem involves translating a written text describing a domain into a database model in either ER or UML notation. Students struggle with the key tasks of identifying important concepts in the domain description and mapping them to the appropriate model constructs.

There have been prior publications on developing modeling tools that support auto-grading [4, 5, 14]. The system demonstrated, AutoER [11], has the following unique contributions:

- Supports students building a design model by directly interacting with the text description rather than drawing model elements.
- Provides precise and customizable auto-marking for UML database design diagrams that exactly matches instructor criteria by utilizing standardized naming of model elements.
- Allows for generation of new questions enabling students to practice modeling.
- System has multiple deployment options. The demo highlights the integration with the PrairieLearn [20] learning management system.

The organization of this paper is as follows. Section 2 provides a background on related work, and Section 3 overviews the AutoER system. The demonstration is described in Section 4, and the paper closes with future work and conclusions.

2 BACKGROUND

There has been extensive work on developing educational systems to automate marking. General systems like PrairieLearn [20] and Web-CAT [10] support arbitrary questions, and there are surveys [8, 16] identifying hundreds of systems for automatic assessment of programming problems. These systems provide real-time, consistent feedback allowing for efficiency for students and instructors.

Several systems have been developed to support the construction and evaluation of database designs in ER or UML class diagrams. Dedicated UML classroom tools like QuickUML [3], minimUML [19], DiagramMER [12], and StudentUML [9] provide domain modeling capabilities with simplified user interfaces compared to industrial software. Surveys [1, 2] of instructional systems for modeling show that students prefer easy to use systems that allow them to focus on learning the concepts rather than the software.

Systems for automatic assessment of design diagrams and models have been developed. A key challenge is handling the variability of student answers and the semantic differences inherent in modeling problems. Systems may be based on image processing [18], comparing models in string form [14, 15, 17], or machine learning

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MODELS '22 Companion, October 23–28, 2022, Montreal, QC, Canada

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9467-3/22/10...\$15.00

<https://doi.org/10.1145/3550356.3559091>

[6]. UMLGrader [14] and UMLint [15] evaluated UML class diagrams developed using IBM Rational Rose and provided feedback to students on errors. [17] presents an UML database design software allowing students to create diagrams with auto-marking. The marking accuracy is typically within a letter grade (i.e. 10 to 15%) and generally similar to the variation in human markers.

Recent work [4] developed an automatic marking system for UML design diagrams using syntactic, structural, and semantic analysis. The system’s auto-grading accuracy was further improved by customizing grading criteria [5] that allowed grades to be within 5 to 15% of instructor grades.

The key challenges with auto-grading models and designs are naming ambiguity and handling different modeling representations. If students are able to use their own names for model constructs, the matching process with instructor answers may have lower precision since it may be unable to identify when the names refer to the same or different constructs. Further, since students may use multiple different modeling concepts, some of which may be equivalent or equally valid solutions, the auto-grader may incorrectly penalize correct submissions or may award marks when the student’s answer is incorrect. Overall, there remains challenges in ensuring accurate marking of models, and any inaccuracy causes issues as students may dispute grades with instructors causing time delays. None of the previous systems considered the task of auto-generating new problems for students to use for practice. It is also valuable for an instructional system to be easily deployed at many institutions.

3 AUTOER SYSTEM

The AutoER system [11] allows for the evaluation and automatic generation of UML database design problems. These problems appear in upper-level courses on databases and software engineering. UML class diagrams represent database entities as classes, relationships as associations, and attributes may be associated with entities and relationships. Evaluation and providing feedback is time-consuming as students may not use the syntax consistently, misinterpret the question, use inconsistent naming and labels, and not understand fundamental modeling constraints.

The AutoER system supports students learning database design diagrams by providing real-time visualization and feedback during question answering. It supports users first learning design concepts and the process of extracting model elements from written text. Students answer instructor-created or randomly generated questions by interacting with the question text to develop the design. Unlike other design software, the student selects the modeling construct to add (entity, attribute, relationship) instead of drawing the construct on the diagram. The diagram is dynamically drawn based on the student answer. The question text contains markup that allows students to select particular words and phrases to add to the diagram. Context-sensitive pop-up menus allow for adding entities and attributes. As the student builds the design, it is displayed and edited by adding and removing entities, attributes, keys, and relationships. The system uses a text representation of the diagram for evaluating and providing feedback.

Figure 1 shows a student interacting with the AutoER system to solve a question. At this point, the user has constructed most of the diagram. By clicking on the word *note*, a pop-up menu is

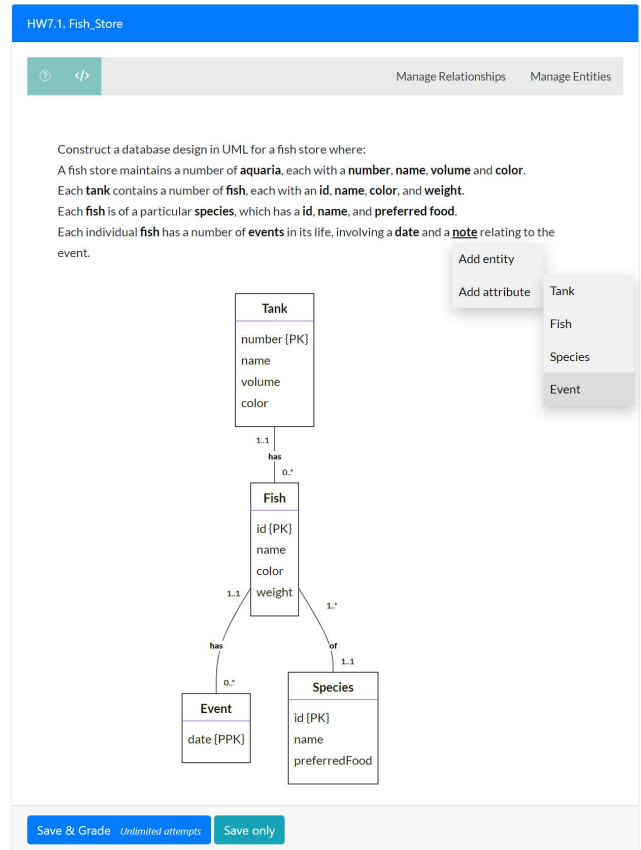


Figure 1: Design Question and UML Class Diagram Answer in AutoER System

shown. The user can add entities and attributes using this pop-up menu. The user is about to add an attribute to the Event entity. The system then updates the diagram to include *note* as an attribute of Event. To add keys to the diagram and modify attributes, the Manage Entities feature is used (see Figure 2). Relationships are managed by using Manage Relationships (see Figure 3). Figure 4 shows the interface for creating a relationship.

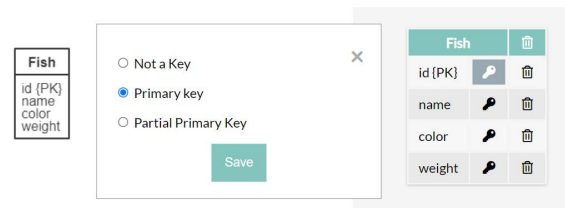


Figure 2: Adding Keys and Managing Entities

Students can submit their answers for grading and feedback at any time (Figure 5). AutoER can be customized on the amount of feedback and the number of attempts allowed per question.

Instructors create questions by adding markup text to highlight clickable phrases to add diagram elements, and providing one or

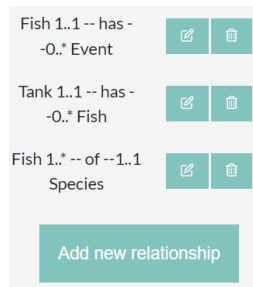


Figure 3: Managing Relationships

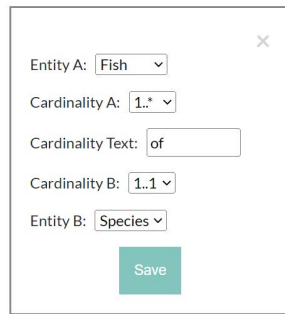


Figure 4: Create Relationship



Figure 5: AutoER System with Feedback Example

more answers in text format. Auto-generated questions are produced by selecting the number of design elements (entities, weak entities, relationships, etc.). The system then randomizes names and relationships. The text question and answer are generated using template sentences. For example, a one-to-many relationship between entities A and B may be described as "A may be related to many B" or "A has multiple relationships with B" or other similar phrases.

The AutoER system was previously evaluated [11] in a large database course at the University of British Columbia. Students overwhelmingly preferred using AutoER compared to the previous design software used in the course (Asth¹). Given the choice of how to build their UML diagrams, 96% used AutoER compared to Asth or manually drawing the diagrams. AutoER was evaluated to

have a System Usability Scale score [7] of 77 indicating overall good usability. Students practiced and submitted more answers to questions. The average grade improved by about 8% on assignments and by about 10% on exams. The system was used during online exams where auto-generated questions prevented student collaboration and copying.

The demo highlights the new integration with PrairieLearn that allows the system to be easily deployed at other institutions. The advantage of using PrairieLearn is its support for many different question types and its customizability to add new questions. Migrating to support PrairieLearn required updating the Python auto-grading code to use the PrairieLearn API. The front-end was implemented as a custom element that can be reused in many courses. Modifications to the user interface were required to support embedding in the PrairieLearn user interface and integration with the PrairieLearn API.

4 DEMONSTRATION

The demonstration shows the key features of the AutoER system including automatic question marking with student feedback, question generation, and integration into the PrairieLearn learning management system.

4.1 Model Marking

The first part of the demonstration shows creation of database design diagrams, and the AutoER system marking and feedback. Publicly available sample problems are at <https://autoed.ok.ubc.ca/demo> for use during the demonstration and freely available for instructors to use in the future. Each database design problem consists of a problem text and one or more correct answers. The demonstration allows participants to build their own models and test them in real-time.

In formative assessments, providing feedback to the user helps them understand and fix their mistakes and improve their learning. For summative assessments, such as exams, AutoER is configurable on the amount of feedback provided, and the number of submissions students can perform.

4.2 Question Generation

Questions are automatically generated by randomly selecting the number of entities, attributes, and relationships in the model and building text that describes the model. The size and complexity of the model built is controllable by the instructor. The text generation may use simple placeholders for entity and attribute names (e.g. A, B, a1, b1, etc.) or gibberish words. In the demonstration, question variants that differ per user are shown.

4.3 PrairieLearn Integration

One of the key challenges with instructors adopting educational systems is the effort in system deployment and customization for their unique environments. The AutoER system is available for public use at <https://autoed.ok.ubc.ca/demo>, and the source code is available to instructors with an open source license. Instructors often need to support integration with the learning management system supported by their institution and require the ability to customize the system for their instructional needs.

¹<https://astah.net/>

This demonstration extends the capabilities of the original, stand-alone AutoER system [11] by integrating with PrairieLearn. The Python code used for question generation and marking has been modified to support the PrairieLearn API. The demo will show how database modeling questions can be created in PrairieLearn, completed by students, and automatically marked by the system.

5 CONCLUSION

Students learn modeling by practice and receiving feedback on their designs. The AutoER system automates the generation and evaluation of database modeling questions to support student learning. The demonstration provides examples on using AutoER on its publicly available website as well as integrating it with learning management systems such as PrairieLearn. The overall goal is to make database modeling easier for instructors to teach students and create public questions for practice use.

Ongoing and future work includes improving the system's capabilities and user interface, including the diagram drawing that uses the open source library Mermaid.js. Additional modeling constructs such as subclasses, methods, and multi-way relationships will be added. Although the current problems are focused on database design that require only a subset of UML, more UML constructs can be added to support more general software engineering and design problems.

6 ACKNOWLEDGMENTS

The authors thank NSERC and UBC for supporting this research. Special acknowledgement to students Emiel van der Poel, Prajeet Didden, Luis Lucio, Siqiao Yuan, Shawn Mountenay, and Reece Walsh for developing and testing the system.

7 APPENDIX

A short video describing the demonstration is available at:

<https://youtu.be/Cxi6Jq4MyDk>.

The demo web site is: <https://autoed.ok.ubc.ca/demo>

REFERENCES

- [1] Luciane Telinski Wiedermann Agner and Timothy C. Lethbridge. 2017. A Survey of Tool Use in Modeling Education. In *20th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, MODELS 2017, Austin, TX, USA, September 17-22, 2017*. IEEE Computer Society, 303–311. <https://doi.org/10.1109/MODELS.2017.1>
- [2] Luciane Telinski Wiedermann Agner, Timothy C. Lethbridge, and Inali Wisniewski Soares. 2019. Student experience with software modeling tools. *Softw. Syst. Model.* 18, 5 (2019), 3025–3047. <https://doi.org/10.1007/s10270-018-00709-6>
- [3] Carl Alphonse and Phil Ventura. 2003. QuickUML: a tool to support iterative design and code development. In *Companion of the 18th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2003, October 26-30, 2003, Anaheim, CA, USA*. ACM, 80–81. <https://doi.org/10.1145/949344.949359>
- [4] Weiyi Bian, Omar Alam, and Jörg Kienzle. 2019. Automated Grading of Class Diagrams. In *22nd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion, MODELS Companion 2019, Munich, Germany, September 15-20, 2019*. IEEE, 700–709. <https://doi.org/10.1109/MODELS-C.2019.00106>
- [5] Weiyi Bian, Omar Alam, and Jörg Kienzle. 2020. Is Automated Grading of Models Effective? Assessing Automated Grading of Class Diagrams. In *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems (MODELS '20)*. ACM, New York, NY, USA, 365–376. <https://doi.org/10.1145/3365438.3410944>
- [6] Younes Boubekeur, Gunter Mussbacher, and Shane McIntosh. 2020. Automatic assessment of students' software models using a simple heuristic and machine learning. In *MODELS '20: ACM/IEEE 23rd International Conference on Model Driven Engineering Languages and Systems, Canada, 18-23 October, 2020, Companion Proceedings*. ACM, 20:1–20:10. <https://doi.org/10.1145/3417990.3418741>
- [7] John Brooke. 1996. SUS: A quick and dirty usability scale. In *Usability Evaluation in Industry*. 189–194.
- [8] Sébastien Combéfis. 2022. Automated Code Assessment for Education: Review, Classification and Perspectives on Techniques and Tools. *Software* 1, 1 (Feb 2022), 3–30. <https://doi.org/10.3390/software1010002>
- [9] Dimitris Dranidis, Ioanna Stamatopoulou, and Marina Ntika. 2015. Learning and Practicing Systems Analysis and Design with StudentUML. In *Proceedings of the 7th Balkan Conference on Informatics Conference (BCI '15)*. ACM, New York, NY, USA, Article 41, 8 pages. <https://doi.org/10.1145/2801081.2801104>
- [10] Stephen H. Edwards and Manuel A. Pérez-Quinones. 2008. Web-CAT: automatically grading programming assignments. In *Proceedings of the 13th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education, ITiCSE 2008*. ACM, 328. <https://doi.org/10.1145/1384271.1384371>
- [11] Sarah Foss, Tatiana Urazova, and Ramon Lawrence. 2022. Automatic Generation and Marking of UML Database Design Diagrams. In *SIGCSE 2022: The 53rd ACM Technical Symposium on Computer Science Education, Providence, RI, USA, March 3-5, 2022, Volume 1*, Larry Merkle, Maureen Doyle, Judith Sheard, Leen-Kiat Soh, and Brian Dorn (Eds.). ACM, 626–632. <https://doi.org/10.1145/3478431.3499376>
- [12] Carlos R. Jaimez González and Jazmin Martínez-Samora. 2020. DiagramMER: A Web Application to Support the Teaching-Learning Process of Database Courses Through the Creation of E-R Diagrams. *iJET* 15, 19 (2020), 4–21. <https://www.online-journals.org/index.php/i-jet/article/view/14745>
- [13] Object Management Group. 2017. OMG® Unified Modeling Language® (OMG UML®). <https://www.omg.org/spec/UML/2.5.1/> Retrieved: 2022-06-05.
- [14] Robert W Hasker. 2011. UMLGrader: an automated class diagram grader. *Journal of Computing Sciences in Colleges* 27, 1 (2011), 47–54.
- [15] Robert W Hasker and Mike Rowe. 2011. UMLint: Identifying defects in UML diagrams. In *2011 ASEE Annual Conference & Exposition*. 22.1558.1 – 22.1558.14.
- [16] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2018. A Systematic Literature Review of Automated Feedback Generation for Programming Exercises. *ACM Trans. Comput. Educ.* 19, 1, Article 3 (sep 2018), 43 pages. <https://doi.org/10.1145/3231711>
- [17] Josep Soler, Imma Boada, Ferran Prados, Jordi Poch, and Ramón Fabregat. 2010. A Formative Assessment Tool for Conceptual Database Design Using UML Class Diagram. *iJET* 5, 3 (2010), 27–33. <https://www.online-journals.org/index.php/i-jet/article/view/1402>
- [18] Pete G. Thomas, Kevin G. Waugh, and Neil Smith. 2006. Using patterns in the automatic marking of ER-diagrams. In *Proceedings of the 11th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education, ITiCSE 2006, Bologna, Italy, June 26-28, 2006*. ACM, 83–87. <https://doi.org/10.1145/1140124.1140149>
- [19] Scott A. Turner, Manuel A. Pérez-Quinones, and Stephen H. Edwards. 2005. minimUML: A minimalist approach to UML diagramming for early computer science education. *ACM J. Educ. Resour. Comput.* 5, 4 (2005), 1:1–1:28. <https://doi.org/10.1145/1186639.1186640>
- [20] Matthew West, Geoffrey L. Herman, and C. Zilles. 2015. PrairieLearn: Mastery-based Online Problem Solving with Adaptive Scoring and Recommendations Driven by Machine Learning. In *122nd ASEE Annual Conference and Exposition*. 26.1238.1 – 26.1238.14. <https://doi.org/10.18260/p.24575>